

2026.04.06.월

🕒 생성일	2026년 4월 6일 오전 9:07
🏷 태그	API

목차

1. API
2. 라우팅과 엔드포인트
3. FastAPI 문서화
4. 데이터 검증 및 변환

▼ 1) API

- 경로 매개변수 (Path Parameters)

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# 여러 매개변수 사용
@app.get("/cafe/{menu_type}/ {str, item_name: str}):
    return {
        "category": menu_type,
        "item": item_name,
        "message": f"{menu_type} 카테고리의 {item_name}을 선택"
    }
```

- 쿼리스트링

```
https://test.com?key1=value1&key2=value2&key3=value3
```

```
https://test.com?keys=value1,value2,value3
```

```
from typing import Optional, List

@app.get("/products-advanced")
def get_products_advanced(
    # 옵션 쿼리
    category: Optional[str] = Query(None, description="상품 카테고리"),
    min_price: Optional[float] = Query(None, ge=0, description="최소 가격"),
    max_price: Optional[float] = Query(None, ge=0, le=1000000, description="최대 가격"),
    # 쿼리 제약조건
    tags: List[str] = Query([], description="상품 태그"),
    in_stock: bool = Query(True, description="재고 여부")
):
    return {
        "category": category,
        "min_price": min_price,
        "max_price": max_price,
        "tags": tags,
        "in_stock": in_stock
    }
```

- 비교

기능	매개변수	쿼리클래스
기본값	가능	가능
타입 검증	가능	가능
숫자 범위 제한	불가능	가능
문자열 길이 제한	불가능	가능
정규표현식 검증	불가능	가능
API 문서 설명	불가능	가능

- RESTful API

```

• @app.get("/api/v1/users")      # 사용자 목록
• @app.post("/api/v1/users")     # 사용자 생성
• @app.get("/api/v1/users/{id}") # 특정 사용자 조회
• @app.put("/api/v1/users/{id}") # 특정 사용자 전체 수정
• @app.patch("/api/v1/users/{id}") # 특정 사용자 부분 수정
• @app.delete("/api/v1/users/{id}") # 특정 사용자 삭제

```

▼ 2) 라우팅과 엔드포인트

- 라우팅: 목적지(URL)을 확인하고 배달하는 역할
- 엔드포인트: URL + HTTP 메서드 조합
- 라우트 그룹화 및 관리

- APIRouter를 이용한 모듈화
- 예시 프로젝트 구조

```

project/
├── main.py
├── routers/
│   ├── _init_.py
│   ├── users.py
│   └── products.py

```

- 경로 변환기 Path()
 - Path() 함수 주요 옵션
 - title: API 문서에 표시될 제목
 - description: 매개변수 설명

- ge, gt: 숫자 최소값 (이상, 초과)
- le, lt: 숫자 최대값 (이하, 미만)
- min_length, max_length: 문자열 길이 제한
- regex: 정규식 패턴 검증

▼ 3) FastAPI 문서화

- Swagger
 - OpenAPI 문서화 도구
 - 시각적 인터페이스
 - API 테스트 기능
 - 상세 API 문서 작성

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

class Book(BaseModel):
    title: str = Field(..., description="도서 제목", example="파이썬 입문서")
    author: str = Field(..., description="저자명", example="김개발")
    isbn: str = Field(..., description="ISBN 번호", example="978-1234567890")
    pages: int = Field(..., description="페이지 수", example=300, gt=0)
    published_date: Optional[datetime] = Field(None, description="출판일")
```

```
class Config:
    schema_extra = {
        "example": {
            "title": "FastAPI 완벽 가이드",
            "author": "이개발자",
            "isbn": "978-1234567890",
            "pages": 450,
            "published_date": "2024-01-15T10:00:00"
        }
    }
```

- 태그 작성

```
tags_metadata = [
    {
        "name": "도서 관리",
        "description": "도서의 등록, 조회, 수정, 삭제 기능을 제공합니다",
        "externalDocs": {
            "description": "도서 관리 매뉴얼",
            "url": "https://example.com/docs/books",
        },
        # externalDocs : 외부 문서 링크
    },
    {
        "name": "사용자 관리",
        "description": "회원 가입, 로그인, 정보 수정 기능을 제공합니다",
    },
    {
        "name": "대출 관리",
        "description": "도서 대출 및 반납 관리 기능을 제공합니다",
    },
]
```

```

app = FastAPI(
    title="도서관 관리 시스템",
    description="도서관 운영을 위한 종합 관리 시스템",
    version="1.0.0",
    openapi_tags=tags_metadata # ← 태그 메타데이터 등록
)

@app.get("/books", tags=["도서 관리"])
def get_books():
    """모든 도서 목록을 조회합니다"""
    return [{"id": 1, "title": "FastAPI 가이드"}]

@app.post("/books", tags=["도서 관리"])
def create_book():
    """새로운 도서를 등록합니다"""
    return {"message": "도서가 등록되었습니다"}

```

- 보안
 - Authentication 인증
 - Authorization 권한
 - Token 토큰

▼ 4) 데이터 검증 및 변환

- 기본 타입 검증
- Field 함수 활용 검증
- 자동 타입 변환
- 커스텀 검증자
- 검증 예러
- 전역 검증 예러 핸들러
- 중첩 모델 활용
- 조건부 검증